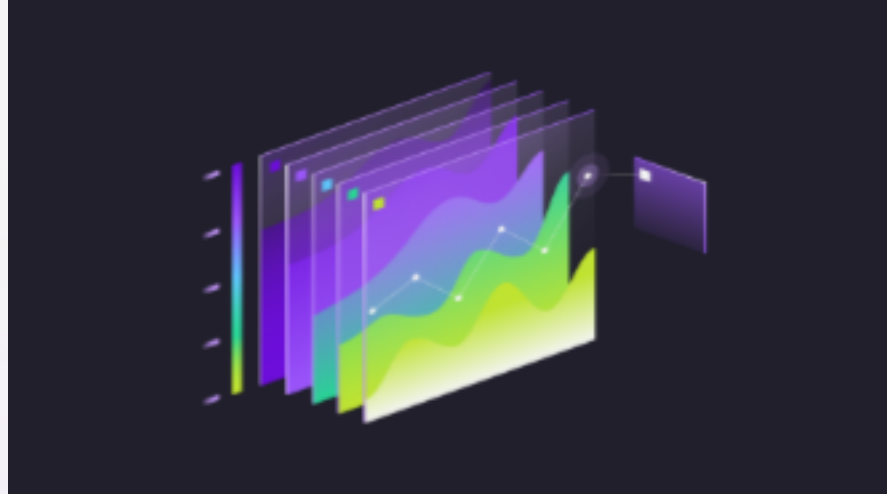




level 2 data collections in zarr

CNG Forum 2026 · Thu, Oct 08, 2026

Sean Harkins (DevSeed) & Tom Nicholas
(Earthmover)





CNG FORUM 2025

last year: zarr vs cog

Lots of debate last year, and the rough consensus was:

LEVEL 3
gridded datacubes

Zarr wins

LEVEL 2
individual scenes

COG + STAC wins

does it have to be that way?



OUTLINE

what we'll cover

1. Desired properties of a cloud-native data system
2. Level 2 vs Level 3 today
3. Limitations of the COG + STAC design
4. Idea: Icechunk DataCollections
5. (Sean) Prototypes and future work



DESIRED PROPERTIES

icechunk-zarr works great! (for level 3)

Property	Icechunk-Zarr
Single endpoint for metadata  	✓
Query by coordinates 	✓
Scalable  	✓
Serverless  	✓
Uncoordinated reads  	✓
Uncoordinated writes 	✓

 i.e. "cloud-optimized"

 from Zarr

Property	Icechunk-Zarr
Arbitrary N-D schemas 	✓
Domain-agnostic 	✓
ACID transactions and consistency 	✓
Versioning, time travel, branches 	✓
Schema evolution 	✓
Zero-copy ingestion ("virtual chunks") 	✓

 from Xarray

 from Icechunk



LEVEL 2 VS LEVEL 3

but level 2 in zarr is unsolved

- You **cannot** think of Level 2 data as one datacube
- The fundamental problem: **many arrays that don't share a coordinate system**
- So geospatial folks use many separate COGs, then index them with STAC
 - We also see anti-patterns: many small Zarrs, or one Zarr with an unwieldy number of groups

Important

This problem is **not** specific to geospatial!

- Bioimaging: millions of OME-TIFFs, plus an index
- Fusion energy: many separate Zarr stores, indexed with Parquet



STAC + COG

solves some problems, but not all

Property	Icechunk-Zarr	STAC + COG
Single endpoint for metadata	✓	✓
Query by coordinates	?	✓
Scalable	✓	✓
Serverless	✓	●
Uncoordinated reads	✓	✓
Uncoordinated writes	✓	●

Property	Icechunk-Zarr	STAC + COG
Arbitrary N-D schemas	✓	✗
Domain-agnostic	✓	✗
ACID transactions and consistency	✓	✗
Versioning, time travel, branches	✓	✗
Schema evolution	✓	✗
Zero-copy ingestion	✓	●



STAC + COG

results in disjoint data systems

- Data lives in one place (COGs in object storage)
- Metadata lives in another (a STAC catalog / database)
- Nothing ties the two together transactionally
- → **Consistency problems** whenever either side changes



IN PRACTICE

these limitations cause real problems

Consistency failures

During operational updates, the catalog and the data can disagree

Rigid data layouts

No hyperspectral bands, no ensemble members, no time dimension

Query-optimized chunking

Can't always rechunk for access patterns, e.g. timeseries

Expensive modifications

Renaming one variable in a 1TB TIFF means downloading and re-uploading 1TB

Domain-specific

Microscopists have near-identical requirements, but COG + STAC are geo-only

Existing archives

Can't reference non-cloud-optimized data, e.g. a biologist's plain TIFFs



WHY

these limitations are inherent to the design

A monolithic container file can never support key features in object storage

- Prevents embarrassingly parallel writes
- Prevents cheap updates → no schema evolution, no versioning
- Can't use the serverless consistency mechanism Icechunk uses
- Virtual references inherently involve multiple files

TIFF as a container constrains the schema

GeoTIFF + STAC are domain-specific



WHAT WOULD BE BETTER?

a domain-agnostic way to manage sets of related arrays

- If only we could put **all the chunks and all the metadata** in one consistently versioned, cloud-native data repository...
- We still want STAC-like metadata search
- But if we can get that *from* Icechunk, we immediately gain loads of powerful features



IDEA

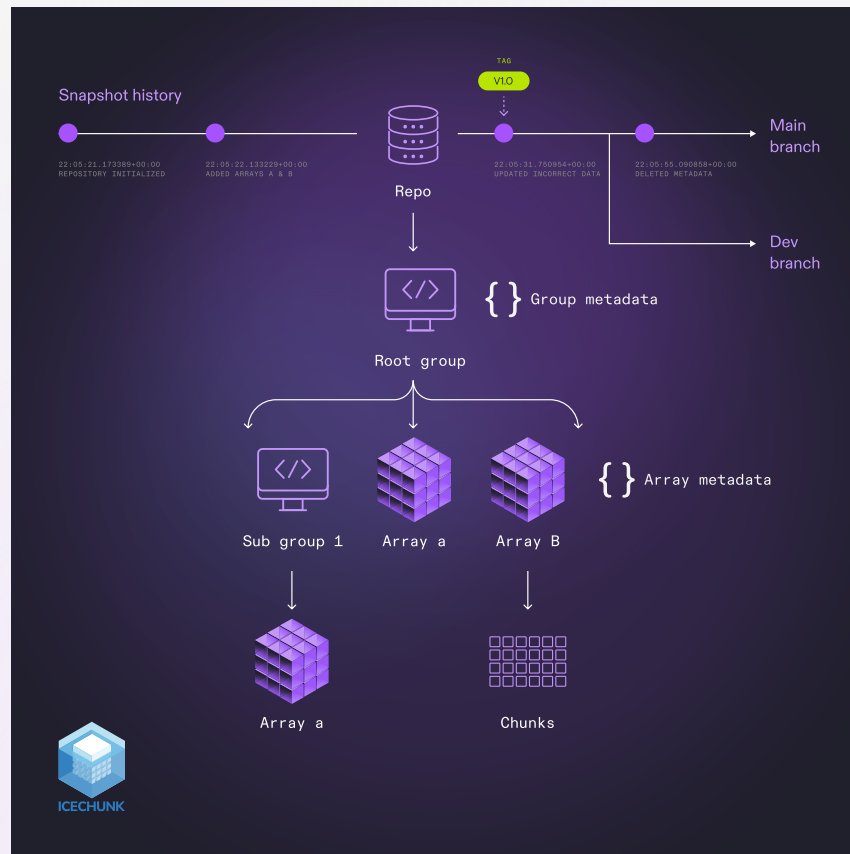
icechunk level 2 datacollections



DATA COLLECTIONS

many images, one repo

- For a Level 3 datacube we usually assume **1 Xarray Dataset == 1 Zarr store**
- What if instead we put **many, many images** into one Zarr store?
- Then we just need a 1D metadata table (which could be STAC-like) to find images of interest
- If that Zarr store is an **Icechunk repo**, we get ACID, consistency, versioning, and virtual chunks — all for free





SEAN HARKINS • DEVELOPMENT SEED

design, prototypes, and future work



earthmover

earthmover.io · developmentseed.org



BACKUP

common objections



"BUT I LIKE DOWNLOADING A SINGLE FILE"

sure — but that needn't dictate the storage layout

- It's reasonable to want a file on your local filesystem
- But we're free to transform the data **any way we like** on its journey from the datacentre to your laptop
- So use a service to subset the cloud-native store and assemble a COG on request
 - e.g. Flux
- A COG that never existed on disk — **the perfect COG for your immediate need**



"BUT A COG IS STILL A VALID TIFF!"

yes — for *filesystem* applications

- We can reconstruct GeoTIFFs on demand for any local program (e.g. ArcGIS)
- It's genuinely impressive that macOS Preview can open a COG — a feat of format stability
 - But no one points Preview at object storage; it can't even read from there
- Any application that reads TIFFs **from object storage** can be taught to read Zarr
 - It all boils down to HTTP range requests anyway
 - Worst case: an adapter layer that makes a Zarr look like a COG (like netcdf-c on Icechunk)
- It's 2026 — that interface layer is a fully specified, verifiable task. Vibe-code it!